

1. Abstract

This document presents the design and implementation of a Mini Security Orchestration, Automation, and Response (SOAR) platform developed in Python as a coursework extension to a previously implemented SNORT-based Network Intrusion Detection System. The platform ingests raw SNORT alert log data, parses each alert into a structured incident record, classifies incident severity according to SNORT priority, executes a rule-based automated response playbook, persists incidents to a SQLite database with duplicate suppression, exposes a Flask-based web dashboard for incident triage and status management, and exports incident data to CSV for reporting purposes. The system is composed of six cooperating modules — `alert_parser.py`, `playbook.py`, `database.py`, `dashboard.py`, `report_generator.py`, and the orchestration entry point `soar.py` — and includes structured, timestamped audit logging of all lifecycle events.

2. Introduction

Security Orchestration, Automation, and Response (SOAR) refers to a class of security operations tooling that connects disparate detection sources, enriches raw alert data into actionable incidents, applies automated or semi-automated response logic, and provides analysts with a centralized interface for incident triage — reducing the manual effort and response latency associated with security operations centre (SOC) workflows. Commercial SOAR platforms (e.g., Splunk SOAR, Palo Alto Cortex XSOAR, IBM Resilient) typically integrate with dozens of detection and enforcement tools; this project implements the conceptual core of that pipeline — parse, classify, respond, record, and report — at a scale suitable for coursework demonstration, using SNORT as the sole upstream alert source.

This project builds directly upon the previously documented SNORT NIDS deployment: where the earlier exercise concluded with SNORT generating raw alert log entries, this Mini SOAR platform begins at that log file and carries each alert through the remainder of the incident lifecycle — from raw text to a triaged, auditable, and reportable security incident record.

3. Objectives

1. To parse unstructured SNORT alert log text into structured incident data using regular-expression extraction.
2. To classify each incident's severity based on its SNORT priority value.
3. To execute an automated response playbook appropriate to each severity level.
4. To persist incidents in a relational database with duplicate-incident suppression.
5. To provide a web-based dashboard for incident visibility, statistics, and status management.
6. To generate exportable CSV incident reports for offline review and audit.

7. To maintain a structured, timestamped audit log of all orchestration and analyst actions.

4. Tools and Environment

Component	Specification
Programming Language	Python 3.x
Web Framework	Flask (see requirements.txt)
Database Engine	SQLite 3 (Python Standard Library — sqlite3)
Core Libraries	re, csv, os, logging (Python Standard Library)
Upstream Alert Source	SNORT 3 alert log (alerts/snort_alerts.log)
Database File	soar_incidents.db
Audit Log Output	logs/soar.log
Report Output	reports/incident_report.csv
Dashboard Address	http://127.0.0.1:5000 (local Flask development server)

As with the earlier coursework components, the implementation favours the Python standard library wherever possible; Flask is the only external dependency required, as declared in requirements.txt.

5. System Architecture

The platform follows a modular pipeline architecture in which each stage of the incident lifecycle is implemented as an independent, single-responsibility module, orchestrated by a central entry point:

Module	Responsibility
alert_parser.py	Reads the raw SNORT alert log and uses regular expressions to extract alert name, priority, timestamp, source IP, and destination IP into structured incident dictionaries.
playbook.py	Classifies incident severity from SNORT priority (1=HIGH, 2=MEDIUM, 3+=LOW) and selects a corresponding simulated automated response action.
database.py	Manages the SQLite incidents table: schema creation, duplicate-checked insertion, retrieval, and status lifecycle updates.
soar.py	Main orchestration engine — sequentially invokes parsing, classification, playbook execution, and storage for every alert, with full audit logging.
dashboard.py	Flask web application exposing a triage dashboard: incident listing, severity statistics, and analyst-driven status updates.
report_generator.py	Exports all stored incidents from SQLite into a timestamped CSV report for offline distribution and audit.

This separation allows each stage to be tested, reused, or replaced independently — for example, `alert_parser.py` could be substituted with a parser for a different NIDS product without requiring any change to `playbook.py`, `database.py`, or `dashboard.py`, since all downstream modules operate on the same structured incident dictionary contract.

6. Methodology and Workflow

6.1 Alert Parsing

`alert_parser.py` reads the complete contents of the SNORT alert log and splits it into individual alert blocks using a blank-line delimiter, reflecting SNORT's default fast-alert output format. Each block is then processed with three targeted regular expressions: one to extract the human-readable alert name from the bracketed SNORT header line, one to extract the numeric priority value, and one combined pattern to extract the timestamp together with the source and destination IPv4 addresses from the network summary line. Incomplete or malformed blocks (fewer than three lines) are skipped defensively, and a block is only converted into an incident dictionary if all three extraction patterns succeed.

6.2 Severity Classification and Playbook Execution

`playbook.py` maps each incident's numeric SNORT priority to a categorical severity level — Priority 1 to HIGH, Priority 2 to MEDIUM, and Priority 3 or greater to LOW — mirroring SNORT's own priority convention in which lower numeric values indicate more severe findings. Based on the resulting severity, `execute_playbook()` selects a corresponding simulated response action: HIGH-severity incidents recommend source IP blocking and analyst notification; MEDIUM-severity incidents recommend watchlisting and further investigation; LOW-severity incidents are recorded for passive monitoring only. The module explicitly simulates these actions and does not perform any live firewall or network enforcement, which is an intentional safety boundary for a coursework environment.

6.3 Duplicate-Checked Persistence

`database.py` persists each incident to a local SQLite database. Before insertion, `save_incident()` queries for an existing row matching on alert name, timestamp, source IP, and destination IP in combination; if a match is found, the incident is treated as a duplicate (for example, from re-running the parser against an unchanged log file) and is silently skipped rather than re-inserted, preventing incident duplication across repeated orchestration runs.

6.4 Orchestration and Audit Logging

`soar.py` acts as the coordinating entry point, executing the pipeline in four sequential stages: database initialization, alert parsing, per-incident processing (classification, playbook execution, and storage), and a final execution summary. Every significant lifecycle event — parse completion, per-alert processing, playbook execution, successful storage, and duplicate skips — is written to a structured, timestamped audit

log (logs/soar.log) using Python's logging module, independently of the console output, providing a durable record suitable for later security review.

6.5 Dashboard Visualisation and Status Management

dashboard.py exposes a Flask web application with two routes: a root dashboard route that retrieves all stored incidents and computes summary counts by severity (HIGH/MEDIUM/LOW) for display, and a status-update route that allows an analyst to transition an incident through its lifecycle (OPEN → INVESTIGATING → RESOLVED). Every status change — successful or failed — is logged to the same shared audit log used by the orchestration engine, giving a unified activity trail across both automated processing and manual analyst action.

6.6 Report Export

report_generator.py provides an offline reporting capability, exporting the full contents of the incidents table to a CSV file (reports/incident_report.csv) with a fixed nine-column header matching the database schema, suitable for import into spreadsheet tools or submission as a coursework audit artefact.

7. Database Schema

The platform uses a single-table SQLite schema (incidents) defined in database.py, structured as follows:

Column	Type	Description
id	INTEGER PRIMARY KEY AUTOINCREMENT	Unique, auto-generated incident identifier.
alert_name	TEXT NOT NULL	Human-readable SNORT alert message (e.g., "ICMP Ping Detected").
priority	INTEGER NOT NULL	Raw SNORT priority value (1 = highest severity).
severity	TEXT NOT NULL	Derived categorical severity: HIGH, MEDIUM, or LOW.
timestamp	TEXT NOT NULL	Alert timestamp as reported by SNORT.
source_ip	TEXT NOT NULL	Source IPv4 address associated with the alert.
destination_ip	TEXT NOT NULL	Destination IPv4 address associated with the alert.
response_action	TEXT NOT NULL	Simulated automated response recommended by the playbook.
status	TEXT DEFAULT 'OPEN'	Incident lifecycle status: OPEN, INVESTIGATING, or RESOLVED.

8. Source Code Implementation

8.1 alert_parser.py — Regex-Based Alert Extraction

The core extraction logic uses three targeted regular expressions applied to the first three lines of each alert block:

```
alert_match = re.search(r"\]\s(.+)\s\[.*\]", lines[0])
priority_match = re.search(r"Priority:\s*(\d+)", lines[1])
network_match = re.search(
    r"(\S+)\s+(\d{1,3})(?:\.\d{1,3}){3}\s+>\s+(\d{1,3})(?:\.\d{1,3}){3})",
    lines[2]
)

if alert_match and priority_match and network_match:
    incident = {
        "alert_name": alert_match.group(1),
        "priority": int(priority_match.group(1)),
        "timestamp": network_match.group(1),
        "source_ip": network_match.group(2),
        "destination_ip": network_match.group(3)
    }
```

8.2 playbook.py — Severity Classification and Response Selection

```
def classify_severity(priority):
    if priority == 1:
        return "HIGH"
    elif priority == 2:
        return "MEDIUM"
    else:
        return "LOW"

def execute_playbook(incident):
    severity = classify_severity(incident["priority"])
    if severity == "HIGH":
        response_action = f"Recommend blocking source IP {incident['source_ip']}..."
    elif severity == "MEDIUM":
        response_action = f"Add source IP {incident['source_ip']} to the watchlist..."
    else:
        response_action = f"Record the alert for source IP {incident['source_ip']}..."
    return {"severity": severity, "response_action": response_action, ...}
```

8.3 database.py — Duplicate-Checked Incident Insertion

```
cursor.execute(
    """SELECT id FROM incidents
       WHERE alert_name = ? AND timestamp = ?
       AND source_ip = ? AND destination_ip = ?""",
    (incident["alert_name"], incident["timestamp"],
     incident["source_ip"], incident["destination_ip"])
)
existing_incident = cursor.fetchone()

if existing_incident:
```

```
print(f"[!] Duplicate incident skipped: {incident['alert_name']}")
return False
```

8.4 soar.py — Orchestration Loop

```
for incident_number, incident in enumerate(incidents, start=1):
    logger.info("ALERT_PROCESSED | ALERT=%s | PRIORITY=%s | ...")
    playbook_result = execute_playbook(incident)
    logger.info("PLAYBOOK_EXECUTED | ALERT=%s | SEVERITY=%s | ACTION=%s")
    was_stored = save_incident(incident, playbook_result)
    if was_stored:
        stored_count += 1
        logger.info("INCIDENT_STORED | ALERT=%s | SOURCE=%s | SEVERITY=%s")
    else:
        logger.warning("INCIDENT_SKIPPED | DUPLICATE | ALERT=%s | SOURCE=%s")
```

8.5 dashboard.py — Flask Route and Status Update

```
@app.route("/update_status/<int:incident_id>/<new_status>", methods=["POST"])
def update_status(incident_id, new_status):
    ...
    was_updated = update_incident_status(incident_id, new_status)
    if was_updated:
        logger.info("STATUS_CHANGED | INCIDENT_ID=%s | ALERT=%s |
                    OLD_STATUS=%s | NEW_STATUS=%s")
    else:
        logger.warning("STATUS_CHANGE_FAILED | INCIDENT_ID=%s |
                      REQUESTED_STATUS=%s")
    return redirect(url_for("dashboard"))
```

9. Configuration and Data Artefacts

9.1 requirements.txt

The project declares a single external dependency, reflecting the minimal-footprint design philosophy applied throughout the coursework:

```
Flask
```

9.2 Generated Runtime Artefacts

At runtime, the platform produces three categories of generated artefacts, none of which are committed as static project files but are instead created programmatically on first execution:

Artefact	Generated By	Purpose
soar_incidents.db	database.py — create_database()	Persistent SQLite store of all parsed and classified incidents.
logs/soar.log	soar.py / dashboard.py — logging config	Structured, timestamped audit trail of orchestration and analyst actions.

Artefact	Generated By	Purpose
reports/incident_report.csv	report_generator.py	Exportable CSV snapshot of all incidents for offline review.

This design — generating rather than statically committing runtime data files — keeps the version-controlled codebase free of environment-specific state and avoids accidentally publishing simulated incident data alongside the source code.

10. Testing and Validation Methodology

Each core module includes a self-contained test harness guarded by an `if __name__ == "__main__":` block, allowing independent validation of every pipeline stage without requiring the full orchestration engine to run:

Module	Standalone Test Behaviour
alert_parser.py	Parses the configured alert log directly and prints every extracted incident field for manual verification against the raw log content.
playbook.py	Executes the playbook against a single hard-coded sample incident (priority 1) and prints the resulting severity and response action.
database.py	Initializes the database schema and prints the full contents of the incidents table, without inserting any sample data.
report_generator.py	Runs <code>generate_incident_report()</code> directly against whatever incidents are currently stored, producing a CSV snapshot for inspection.

End-to-end validation is performed by executing `soar.py`, which exercises the complete pipeline — parsing, classification, playbook execution, duplicate-checked storage, and audit logging — in a single run, followed by launching `dashboard.py` to confirm that stored incidents render correctly and that status transitions are correctly persisted and logged.

11. Security Considerations and Discussion

- **Simulated Response Actions:** `playbook.py` deliberately does not perform live enforcement (e.g., firewall rule changes) — response actions are descriptive recommendations only, which is an appropriate and safe boundary for a coursework SOAR implementation, avoiding unintended network impact.
- **Duplicate Suppression Design:** the duplicate check in `save_incident()` relies on an exact match across four fields (alert name, timestamp, source IP, destination IP). Because SNORT timestamps are typically unique to the microsecond, this design reliably prevents duplicate storage on repeated

parser runs against an unchanged log file, while still allowing genuinely distinct alerts with the same name and IPs (but different timestamps) to be recorded separately.

- **Audit Trail Integrity:** both automated orchestration events and manual analyst status changes are written to a single shared log file (`logs/soar.log`), providing a unified, chronologically ordered record — an important property for later incident review or compliance reporting.
- **Local-Only Dashboard Exposure:** the Flask dashboard is bound explicitly to `127.0.0.1`, restricting access to the local host and avoiding inadvertent exposure of incident data on the network during coursework development and testing.

12. Limitations and Future Enhancements

- The alert parser is coupled to the specific three-line structure of SNORT's default fast-alert output format and would require adaptation to ingest alerts from other NIDS engines or SNORT's alternative output formats (e.g., `unified2`).
- Response playbooks are simulated only; extending the platform toward genuine automated response (e.g., firewall API integration) would require careful additional safeguards, authentication, and change-approval workflows before any production use.
- The dashboard currently lacks analyst authentication, meaning any local user with browser access to `127.0.0.1:5000` can change incident status; a production deployment would require login-gated access control.
- Severity classification is derived solely from SNORT's static priority field; a more advanced implementation could incorporate contextual enrichment (e.g., asset criticality, threat intelligence feeds) into the classification logic.
- The SQLite backend is well suited to single-analyst coursework use but would need to be replaced with a concurrent-access database (e.g., PostgreSQL) to support a multi-analyst SOC environment.

13. Conclusion

This project successfully implements the core stages of a Security Orchestration, Automation, and Response pipeline — alert parsing, severity classification, automated (simulated) response, persistent duplicate-checked storage, web-based triage, and CSV reporting — as a natural extension of the earlier SNORT NIDS coursework. The modular six-file architecture demonstrates sound separation of concerns, and the consistent use of structured audit logging across both automated and analyst-driven actions reflects real-world SOC operational requirements. Collectively, the exercise provides practical, hands-on grounding in the design patterns that underpin commercial SOAR platforms, while maintaining safe, non-destructive behaviour appropriate to a coursework environment.